

QEMU/KVM y libvirt

Conceptos, instalación y gestión de máquinas virtuales

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 IV · Infraestructura Virtual

IV · QEMU/KVM Y LIBVIRT

01

QEMU/KVM y libvirt

Virtualización completa en Linux

QEMU

" ***QEMU** es un emulador genérico y de código abierto de máquinas virtuales.*

DOS MODOS DE FUNCIONAMIENTO

MODO EMULADOR

- Permite ejecutar SO de una **arquitectura distinta** (ej. ARM sobre x86)
- Útil para **desarrollo cruzado**
- Rendimiento bajo

MODO VIRTUALIZACIÓN

- Apoyado en hipervisores como **KVM**
- Aprovecha las extensiones del procesador
- **Alto rendimiento**

KVM

*" Kernel-based Virtual Machine es un hipervisor de **tipo 1** integrado al kernel de Linux.*

CARACTERÍSTICAS

- Solución de **virtualización completa** para Linux
- Necesita CPU con extensiones **Intel VT** o **AMD-V**
- Se compone de varios **módulos del kernel**:
 - `kvm.ko` — infraestructura base de virtualización
 - `kvm-intel.ko` / `kvm-amd.ko` — módulo específico del procesador

 **QEMU + KVM** es la combinación habitual: QEMU emula los dispositivos y KVM acelera la ejecución del invitado.

Dispositivos paravirtualizados (`virtIO`)

- En virtualización completa, los dispositivos (discos, red...) están **emulados por software**: el invitado cree que habla con hardware real, y cada operación se traduce mediante una capa de emulación
- Un dispositivo **paravirtualizado** es distinto: el invitado *sabe* que está virtualizado y usa un **driver específico** que habla directamente con el hipervisor mediante una interfaz simple y eficiente, sin fingir ser hardware real
- KVM agrupa estos dispositivos bajo el estándar `virtIO`

DISPOSITIVOS EMULADOS

- Compatibilidad universal
- **Rendimiento bajo**
- Cada operación atraviesa la capa de emulación

DISPOSITIVOS `VIRTIO`

- `virtio-net` — tarjeta de red
- `virtio-blk` / `virtio-scsi` — disco
- **Rendimiento muy cercano al real**

libvirt

*" libvirt es la API y conjunto de herramientas que facilita la **gestión** de los recursos virtualizados.*

¿POR QUÉ LIBVIRT?

- Trabajar directamente con QEMU/KVM es **complejo**
- libvirt ofrece una **API genérica** y un **demonio** comunes
- Soporta varios sistemas: **KVM, LXC, Xen...**
- Permite usar las **mismas herramientas** independientemente del hipervisor

Mecanismos de conexión a libvirt

LOCAL SIN PRIVILEGIOS

```
qemu:///session
```

- Acceso a las VM **del usuario actual**
- Sin permisos para crear redes
- Útil para usuarios de escritorio

LOCAL PRIVILEGIADO

```
qemu:///system
```

- Acceso a las VM **del sistema**
- Permisos completos sobre red y almacenamiento

REMOTO PRIVILEGIADO POR SSH

```
qemu+ssh:///system
```

- Conexión **a un servidor remoto** que ejecuta libvirt
- Autenticación a través de **SSH**
- Base para administrar **clústeres** de hipervisores

Aplicaciones del ecosistema libvirt

APLICACIÓN	FUNCIÓN
virsh	Cliente oficial de línea de comandos . Shell completa para la API
virt-manager	Aplicación gráfica con la mayor parte de las funcionalidades
virt-install	Creación de MV desde la línea de comandos (<code>virt-install</code> , <code>virt-clone</code> , <code>virt-xml</code>)
virt-viewer	Acceso a la consola gráfica de una VM
gnome-boxes	Aplicación gráfica simple para usuarios de escritorio



`virsh` es la herramienta de referencia para automatizar y administrar libvirt en servidores sin entorno gráfico.

IV · QEMU/KVM Y LIBVIRT

02

Instalación y primeros pasos

Instalación, red y almacenamiento por defecto

Instalación de QEMU/KVM + libvirt

```
root@kvm:~# apt install qemu-system libvirt-clients libvirt-daemon-system  
  
root@kvm:~# adduser usuario libvirt  
  
usuario@kvm:~$ virsh -c qemu:///system list
```

Para no tener que indicar la conexión en cada comando, podemos definir una variable de entorno:

```
export LIBVIRT_DEFAULT_URI='qemu:///system'
```

 Con la variable definida, todos los comandos `virsh` usarán `qemu:///system` sin especificarlo.

Red disponible por defecto

```
usuario@kvm:~$ virsh net-list --all
Nombre      Estado      Inicio automático  Persistente
-----
default     inactivo    no                  si

usuario@kvm:~$ virsh net-start default
La red default se ha iniciado

usuario@kvm:~$ virsh net-autostart default
La red default ha sido marcada para iniciarse automáticamente
```

Características de la red `default`

La red `default` es de tipo **NAT**. Al crear una MV se conectará a ella por defecto:

1. El host tiene un **servidor DHCP** (rango: `192.168.122.2 - 192.168.122.254`)
2. La **puerta de enlace** de la MV es `192.168.122.1` (el propio host)
3. El **servidor DNS** de la MV también es el host
4. La MV se conecta a un **Linux Bridge** llamado `virbr0`
5. El host se conecta al bridge `virbr0` con la IP `192.168.122.1`
6. El host realiza **SNAT** para dar conectividad al exterior

Red default — esquema

```
prueba1 (1) - Virt Viewer
Fichero Ver Enviar llave Ayuda
usuario@prueba1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 52:54:00:8a:50:d1 brd ff:ff:ff:ff:ff:ff
    inet 192.168.122.39/24 brd 192.168.122.255 scope global dynamic enp1s0
        valid_lft 2859sec preferred_lft 2859sec
    inet6 fe80::5054:ff:fe8a:50d1/64 scope link
        valid_lft forever preferred_lft forever
usuario@prueba1:~$ ip r
default via 192.168.122.1 dev enp1s0
192.168.122.0/24 dev enp1s0 proto kernel scope link src 192.168.122.39
usuario@prueba1:~$ cat /etc/resolv.conf
nameserver 192.168.122.1
usuario@prueba1:~$ ping www.juntadeandalucia.es
PING www.juntadeandalucia.es (217.12.30.81) 56(84) bytes of data.
64 bytes from 81.zone-217.12.30.juntadeandalucia.es (217.12.30.81): icmp_seq=1 ttl=44 time=43.9 ms
64 bytes from 81.zone-217.12.30.juntadeandalucia.es (217.12.30.81): icmp_seq=2 ttl=44 time=43.9 ms
^C
--- www.juntadeandalucia.es ping statistics ---
```

Almacenamiento disponible por defecto

- Los discos de las MV se guardan por defecto en **ficheros con formato** `qcow2`
- El directorio de almacenamiento es `/var/lib/libvirt/images`

```
prueba1 (1) - Virt Viewer
```

Fichero	Ver	Enviar llave	Ayuda
---------	-----	--------------	-------

```
usuario@prueba1:~$ lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
sr0	11:0	1	1024M	0	rom	
vda	254:0	0	10G	0	disk	
├─vda1	254:1	0	9G	0	part	/
├─vda2	254:2	0	1K	0	part	
└─vda5	254:5	0	975M	0	part	[SWAP]

```
usuario@prueba1:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	976Mi	67Mi	825Mi	0,0Ki	83Mi	797Mi
Swap:	974Mi	0B	974Mi			

```
usuario@prueba1:~$ _
```

virt-install

Crea una MV desde la línea de comandos (`apt install virtinst`):

```
virt-install \  
    --virt-type kvm \  
    --name prueba1 \  
    --cdrom ~/iso/debian-11.3.0-amd64-netinst.iso \  
    --os-variant debian10 \  
    --disk size=10 \  
    --memory 1024 \  
    --vcpus 1
```

Para acceder a la consola gráfica: `virt-viewer prueba1`

Gestión de MV con virsh

```
virsh --help  
virsh list --help
```

Subcomandos más usados:

```
list --all  
shutdown <máquina>  
start <máquina>  
autostart <máquina>  
reboot <máquina>  
destroy <máquina>  
suspend <máquina>  
resume <máquina>  
undefine --remove-all-storage <máquina>  
dominfo <máquina>  
domifaddr <máquina>  
domblklist <máquina>
```

Definición XML de una máquina

```
virsh dumpxml <máquina>
```

```
<domain type='kvm' id='6'>
  <name>prueba1</name>
  <uuid>a88eebdc-8a00-4b9d-bf48-cbed7bb448d3</uuid>
  ...
  <memory unit='KiB'>1048576</memory>
  <currentMemory unit='KiB'>1048576</currentMemory>
  <vcpu placement='static'>1</vcpu>
  ...
  <os>
    <type arch='x86_64' machine='pc-q35-5.2'>hvm</type>
    <boot dev='hd' />
  </os>
```

Definición XML — disco e interfaz de red

```
<disk type='file' device='disk'>
  <driver name='qemu' type='qcow2' />
  <source file='/var/lib/libvirt/images/prueba1.qcow2' />
  <target dev='vda' bus='virtio' />
  <address type='pci' domain='0x0000' bus='0x04' slot='0x00' function='0x0' />
</disk>
...
<interface type='network'>
  <mac address='52:54:00:8a:50:d1' />
  <source network='default' />
  <model type='virtio' />
  <address type='pci' domain='0x0000' bus='0x01' slot='0x00' function='0x0' />
</interface>
```



El disco y la tarjeta de red usan el bus **virtio** para mayor rendimiento (dispositivos paravirtualizados).

Modificación de una máquina virtual

Siempre que exista un subcomando específico de `virsh`, es preferible usarlo antes que editar el XML a mano: valida los cambios y reduce el riesgo de dejar la configuración en un estado inconsistente.

COMANDOS VIRSH (RECOMENDADO)

```
# Renombrar (MV parada)
virsh domrename prueba2 prueba1

# Cambiar vCPUs (MV parada)
virsh setvcpus prueba1 2 --config

# Arranque automático
virsh autostart prueba1
```

EDICIÓN XML DIRECTA

```
virsh edit prueba1
```

Abre el XML en `$EDITOR`. Solo para cambios sin subcomando equivalente.



Algunos cambios requieren la MV **parada**, otros admiten **cambio en caliente** y otros necesitan **reinicio**.

Modificación de memoria

Con la MV **parada**, editando el XML:

```
virsh edit prueba1
...
<memory unit='KiB'>3145728</memory>
<currentMemory unit='KiB'>1048576</currentMemory>
...
```

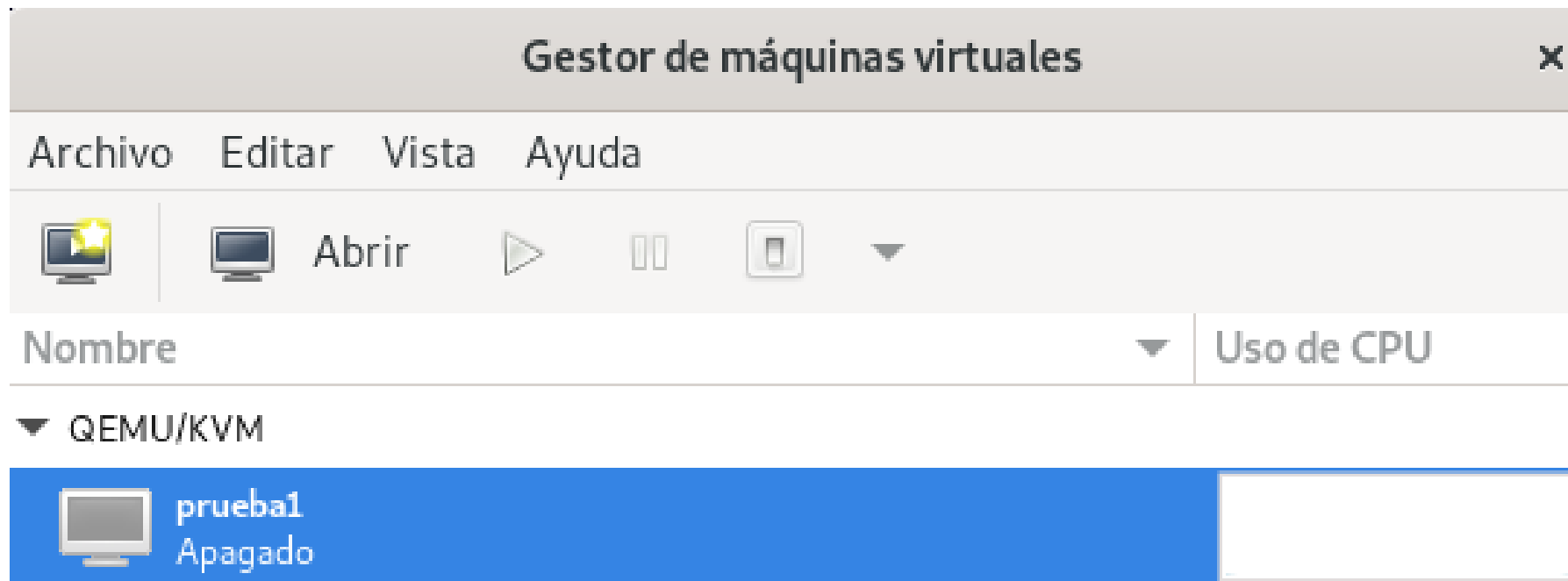
O **en caliente** con la MV arrancada:

```
virsh start prueba1

virsh setmem prueba1 2048M
```

virt-manager

Aplicación gráfica para gestionar libvirt:



Creación de MV Windows — el problema

- Por defecto, `virt-install` puede crear discos y tarjetas de red **sin especificar el bus**: usa dispositivos **emulados** (ej. disco IDE, red `e1000`), que funcionan sin drivers pero con **peor rendimiento**
- Si se piden dispositivos **VirtIO** (`bus=virtio`, `model=virtio`) para mejor rendimiento, Windows **no los reconoce de forma nativa**: el instalador ni siquiera detecta el disco
- Hace falta instalar los **drivers VirtIO para Windows** durante la propia instalación

 Con `bus=virtio` y sin proporcionar los drivers, el instalador de Windows no encuentra ningún disco donde instalar el sistema.

Creación de MV Windows — drivers VirtIO

1. Descarga la ISO de drivers del proyecto oficial **virtio-win**: <https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/>
2. Añádela como un **segundo CD-ROM** en la propia creación de la MV
3. Durante la instalación, cuando no aparezca ningún disco, usa "**Cargar controlador**" y navega hasta ese CD-ROM

```
virt-install \
    --virt-type kvm \
    --name prueba4 \
    --cdrom ~/iso/Win10_21H2_Spanish_x64.iso \
    --os-variant win10 \
    --disk size=40,bus=virtio \
    --disk ~/iso/virtio-win-0.1.217.iso,device=cdrom \
    --network=default,model=virtio \
    --memory 2048 \
    --vcpus 2
```

Para profundizar

- **Curso 1:** Introducción a la virtualización con KVM/libvirt usando virt-manager
github.com/josedom24/curso_kvm_ow/curso1
- **Curso 2:** Profundización en la virtualización con KVM/libvirt
github.com/josedom24/curso_kvm_ow/curso2

IV · QEMU/KVM Y LIBVIRT

¡Gracias!

QEMU/KVM y libvirt

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 IV